

Quantum Frameworks and Libraries

1 Understanding the Need

So, back in the day when tigers used to smoke, some swine decided to invent math, and all the people in the world had to sludge away adding and multiplying numbers all day long.

Until, to fend off this great evil from the world, calculators were born.

The same is true for quantum computing. If you are reading this, I'm assuming you know what a qubit or a quantum gate is by now. But imagine having to define them every single time you need them. That would be fairly annoying. So people decided to create Frameworks and Libraries to make this work a little less tedious and annoying. They did this free of cost and made it publicly available. (Open Source for the win!!)

In this text, we'll learn about these great people and their work. (Okay, maybe just the work because I'm too lazy to find out about these people)

2 Getting familiar with the Libraries

Libraries are pieces of code you can import into your own and use without exactly understanding what is going on under the hood. They make life easy for all of us, abstracting away all unnecessary complications.

Here, we will primarily discuss two quantum computing frameworks in Python, namely Qiskit and PennyLane.

The primary difference between the two is the amount of complexity in their use and the amount of control they allow you over your circuits.

Qiskit provides incredible speed and control over what you build, but it can be overly complex for beginners.

PennyLane, on the other hand, is easier to learn and implement, but is slower and allows less control. We'll be using both to build things, depending on our needs and use case.

2.1 Implementing a simple circuit using PennyLane

2.1.1 Installing PennyLane

Ensure you have Python 3.11+ installed on your system.

Open the terminal in your desired directory and run the following commands:

```
1 #create a virtual environment named .venv
2 python -m venv .venv
3 #install the latest version of PennyLane
4 pip install pennylane
```

Voila! You have now installed PennyLane.

2.1.2 Building a simple Quantum Circuit

Create a new Python file in your directory named `first_qc.py`

In that file, copy and paste the code below:

```
1 import pennylane as qml
2
3 # 1. Initialize a 1-qubit device
4 dev = qml.device('default.qubit', wires=1)
5
6 # 2. Create the QNode
7 @qml.qnode(dev)
8 def first_circuit():
9     # Qubits start in state |0>.
10    # The Pauli-X gate flips the qubit on wire 0 from |0> to |1>
11    qml.PauliX(wires=0)
12
13    # Return the measurement probabilities for states |0> and |1>
14    return qml.probs(wires=0)
15
16 # 3. Execute the circuit
17 result = first_circuit()
18
19 print(f"Probability of measuring |0>: {result[0]}")
20 print(f"Probability of measuring |1>: {result[1]}")
```

Let's understand this code line by line.

```
1 import pennylane as qml
```

This line tells our code to fetch the PennyLane library and alias it as `qml` (by convention).

```
1 dev = qml.device('default.qubit', wires=1)
```

This line creates the virtual hardware where our quantum circuit will run.

```
1 @qml.qnode(dev)
```

This is a Python decorator. Think of it as a bridge between Python and PennyLane, telling it that the following function will be a quantum circuit.

```
1 def first\_circuit():
```

This creates a Python function called `first_circuit` with no parameters. The previous line tells Python that this function is a Quantum circuit meant to run on the hardware `dev`

```
1 qml.PauliX(wires=0)
```

This applies a PauliX Gate to our qubit. It is effectively a NOT Gate in quantum computing. Since qubits are $|0\rangle$ by default, we flipped our qubit into $|1\rangle$

```
1 return qml.probs(wires=0)
```

This line measures the probability of our qubit being 0 or 1 and returns it as a tuple.

```
1 result = first\_circuit()
```

Here, we tell Python to execute the circuit we just created.

```
1 print(f"Probability of measuring |0>: \{result[0]\}")
2 print(f"Probability of measuring |1>: \{result[1]\}")
```

At last, we print our outputs.

If you followed what we discussed so far, you should be able to guess the output. Run the following command in your terminal to verify the results!

```
1 .venv/Scripts/Activate
2 python first_circuit.py
```

2.2 Implementing a simple circuit using Qiskit

2.2.1 Installing Qiskit

Ensure you have Python 3.7+ installed on your system.

Open the terminal in your desired directory and run the following commands:

```
1 #create a virtual environment named .venv
2 python -m venv .venv
3 #install the latest version of Qiskit
4 pip install qiskit
```

Voila! You have now installed Qiskit.

2.2.2 Building a simple Quantum Circuit

Create a new Python file in your directory named `first_qc.py`

In that file, copy and paste the code below:

```
1 from qiskit import QuantumCircuit
2 from qiskit.quantum_info import Statevector
3
4 # 1. Create a Quantum Circuit with 1 qubit
5 qc = QuantumCircuit(1)
6
7 # 2. Apply the Pauli-X (NOT) gate to the first qubit
8 qc.x(0)
9
10 # 3. Simulate the exact quantum state
11 state = Statevector(qc)
12
13 # 4. Extract and print the measurement probabilities
14 probs = state.probabilities_dict()
15 print(f"Measurement probabilities: {probs}")
```

Let's understand this code line by line:

```
1 from qiskit import QuantumCircuit=
2 from qiskit.quantum\_info import Statevector
```

We import the two core tools we need from Qiskit. `QuantumCircuit` is the foundation of every Qiskit program, used to build the step-by-step instructions. `Statevector` is a mathematical tool that lets us perfectly simulate the quantum state of our circuit without needing to connect to actual hardware.

```
1 qc = QuantumCircuit(1)
```

This line creates a quantum circuit with 1 qubit, just like PennyLane; all qubits in Qiskit are initialized as $|0\rangle$

```
1 qc.x(0)
```

This applies the PauliX Gate to our qubit.

```
1 state = Statevector(qc)
```

This line mathematically calculates the exact, idealized state of the qubit after the gates have been applied.

```
1 probs = state.probabilities\_dict()
```

`print(f"Measurement probabilities: {probs}")` These lines compute the probabilities that our qubit is 0 or 1 and print them.

If you followed what we discussed so far, you should be able to guess the output. Run the following command in your terminal to verify the results!

```
1 .venv/Scripts/Activate
2 python first_circuit.py
```

3 Further Reading

If you enjoyed learning about these libraries so far and are curious to know more, I have listed some resources below:

1. Qiskit Documentation: <https://quantum.cloud.ibm.com/docs/en/guides>
2. PennyLane Documentation: <https://docs.pennylane.ai/en/stable/>
3. Coding with Qiskit: https://youtube.com/playlist?list=PL0FEBzvs-VvrgHZt3exM_NNiNKtZlHvZi&si=jcZ1lJFWVGM1YQ21
4. PennyLane Tutorial: <https://youtube.com/playlist?list=PLzgi0kRtN5s08dkomgshjSGDabnjtjBiA&si=S0agCA4Xd-hcd0VV>